

Computer Networks Mastery

From the OSI model to the FAANG networking round
9 diagrams, 17 model answers, a recently-asked set, 50 FAQs

[@rathoreadiitya](#)

v1.0 · Updated 2026-06-06

Brand new to CN? Start at **§3 Start here: the student minimum** below, then read the core concepts in order.

Two readers, one sheet: a 2nd-year who has never heard of a 3-way handshake, and a final-year about to face a networking round at Microsoft or a service company. Read top to bottom if CN is new. Short on time? Use the table of contents to jump.

Inside this sheet

1. Brand-new note + who this sheet is for
2. Inside this sheet (you are here)
3. Start here: the student minimum - what a CN round really tests
4. Why CN feels like memorization (and how to fix that)
5. The OSI model - 7 layers, what each one does
6. TCP/IP model + OSI vs TCP/IP
7. Encapsulation - how a message becomes a frame on the wire
8. Devices - hub vs switch vs router (and which layer)
9. IP addressing - IPv4, classes, private ranges, IPv6
10. Subnetting + CIDR - the math, worked
11. TCP vs UDP - the comparison interviewers always ask
12. TCP connection - 3-way handshake + 4-way teardown
13. Beyond the basics (advanced, not all fresher-required)
14. Flow control vs congestion control
15. Sliding window - Stop-and-Wait, Go-Back-N, Selective Repeat
16. DNS - turning a name into an IP
17. HTTP + HTTPS - methods, status codes, versions
18. TLS handshake - how HTTPS stays secure
19. ARP, DHCP, NAT, ICMP - the supporting cast
20. What happens when you type google.com (the full journey)
21. Error detection + switching (brief)
22. Comparison tables (the ones interviewers love)
23. Interview triggers - if they ask X, say Y
24. Common mistakes + traps
25. Model interview answers - say it like this (17 questions)
26. Recently asked in interviews - what current cycles hit
27. Interview FAQ bank - 50 questions
28. The 30-minute revision card
29. Final word

If you have only 30 minutes - jump to §28 Revision card.

3. Start here: the student minimum

A CN round for a fresher is not the whole textbook. It is a small core asked over and over. Learn this core cold and you will not drop easy points in most rounds.

The six must-knows:

1. The OSI layers and the TCP/IP model - the order, and one job per layer.
2. TCP vs UDP - connection-oriented + reliable vs connectionless + fast, and which apps use each.
3. The TCP 3-way handshake - SYN, SYN-ACK, ACK - and why it is three, not two.
4. IP addressing + a basic subnetting calculation - hosts in a /26, private ranges.
5. DNS - how google.com becomes an IP address.
6. What happens when you type a URL and press Enter - the end-to-end story that ties everything together.

The one thing to be able to narrate cold: what happens when you type google.com and hit Enter. DNS lookup, TCP handshake, TLS, HTTP request, response, render. If you can walk that path without notes, you have touched almost every CN topic at once. It is the single most asked networking question.

Where to jump: OSI is §5, TCP vs UDP is §11, the handshake is §12, subnetting is §10, DNS is §16, and the full URL journey is §20. Everything before §13 is the fresher core. Everything after is depth that separates a good answer from a great one.

Major sections carry a difficulty badge so you know where to spend your time:

 **Must know** - asked in ~80% of rounds, any company.  **Good to know** - mostly product / top companies.  **Rare / deep dive** - low-yield for freshers, do not stress.

4. Why CN feels like memorization

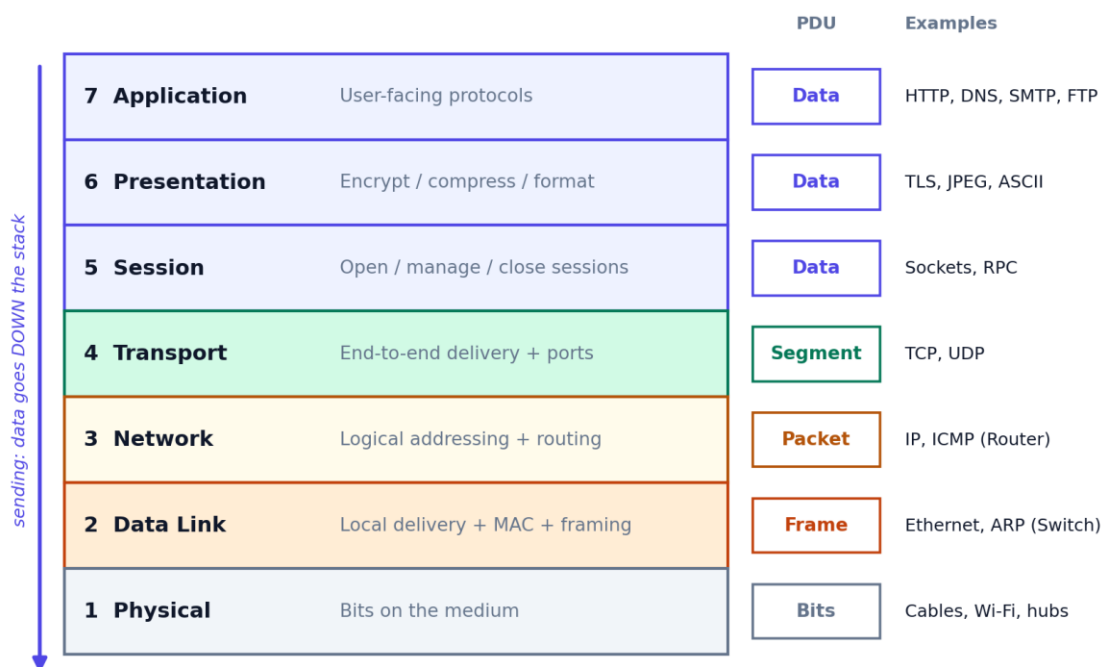
CN looks like a pile of acronyms - OSI, TCP, UDP, DNS, ARP, DHCP, NAT. Students try to rote-learn them and forget everything a week later. The fix is to see the network as a postal system: an address to find the destination (IP), a street-level handoff (MAC), a reliable courier who gets a signature (TCP) or a cheap postcard with no guarantee (UDP), and a phone book that turns names into addresses (DNS). Map each acronym to one everyday scene and it stops being trivia. That is what the Aha boxes in this sheet do.

5. The OSI model - 7 layers

 **Must know**

The OSI model is a 7-layer reference map. Data starts at the top (the app), travels down adding a header at each layer, crosses the wire, then travels up the other side stripping headers off. You do not need to memorize every detail - you need the order, one job per layer, and the PDU (the unit of data) at each.

The OSI model: 7 layers, top (apps) to bottom (wires)



Mnemonic (top to bottom): All People Seem To Need Data Processing.

The 7 layers, their one-line job, the PDU, and example protocols. Mnemonic top to bottom: All People Seem To Need Data Processing.

Layer	Job in one line	PDU	Examples
7 Application	Protocols the user's app speaks	Data	HTTP, DNS, SMTP, FTP
6 Presentation	Encrypt, compress, translate format	Data	TLS, JPEG, ASCII
5 Session	Open, manage, close conversations	Data	Sockets, RPC
4 Transport	End-to-end delivery + ports	Segment	TCP, UDP
3 Network	Logical addressing + routing	Packet	IP, ICMP
2 Data Link	Local delivery via MAC + framing	Frame	Ethernet, ARP
1 Physical	Raw bits on the medium	Bits	Cables, Wi-Fi, hubs

💡 Aha - OSI ko ek courier company ki tarah socho. Application layer = tumne letter likha. Transport = courier guarantee deta hai ki pahunchega (TCP) ya bina guarantee daal deta hai (UDP). Network = pin code se city dhoondhna (IP). Data Link = mohalle mein ghar tak haath se dena (MAC). Physical = woh road jis par truck chalti hai.

The layers 5, 6, 7 are often merged into a single Application layer in practice (that is exactly what the TCP/IP model does). For a fresher round, the four that carry the real weight are Transport, Network, Data Link, and Application.

6. TCP/IP model + OSI vs TCP/IP

● Must know

The OSI model is the teaching map. The TCP/IP model is what the actual internet runs on. It collapses OSI's 7 layers into 4 by merging the top three into one Application layer and the bottom two into one Link layer.

TCP/IP layer	Maps to OSI	Does what	Protocols
Application	5 + 6 + 7	App protocols + format + sessions	HTTP, DNS, FTP, SMTP
Transport	4	End-to-end delivery + ports	TCP, UDP
Internet	3	Addressing + routing across networks	IP, ICMP, ARP
Link / Network access	1 + 2	Local delivery + the physical medium	Ethernet, Wi-Fi

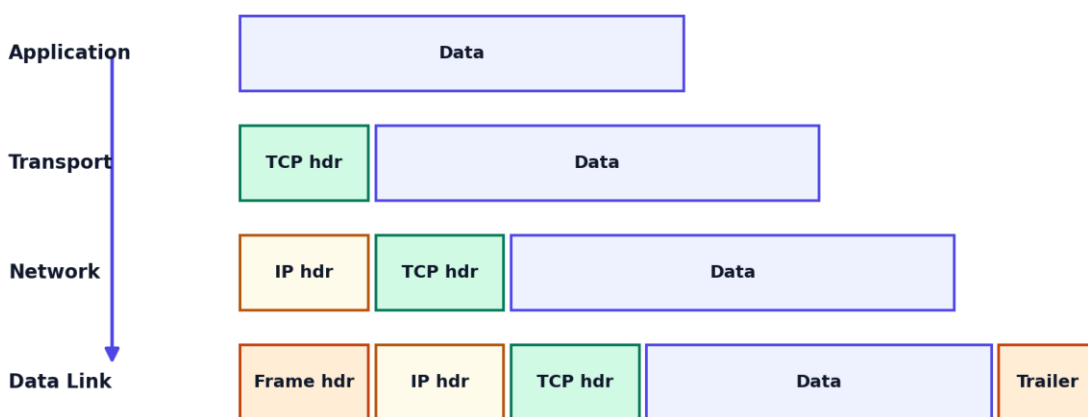
💡 Aha - OSI textbook ka detailed map hai 7 layers ka, TCP/IP asli zindagi ka chhota 4-layer map hai. Jaise syllabus mein 7 chapters hain par exam mein 4 hi unit aate hain - kaam wahi hai, bas group kar diya.

7. Encapsulation - message to frame

● Good to know

As data goes down the stack, every layer wraps the chunk from above in its own header (and the Data Link layer adds a trailer too). The receiver does the reverse - it peels one header off at each layer on the way up. This is why the same data is called a segment at Transport, a packet at Network, and a frame at Data Link.

Encapsulation: every layer wraps the one above in its own header



The Data Link frame is what actually goes on the wire. The receiver peels each header off on the way up.

Each layer adds its header. The Data Link frame is what physically travels on the wire.

💡 Aha - Encapsulation gift wrapping jaisa hai. Andar ka gift same hai, par har layer ek naya wrapper chadhati hai - transport ka, network ka, data link ka. Receiver ek-ek wrapper khol ke

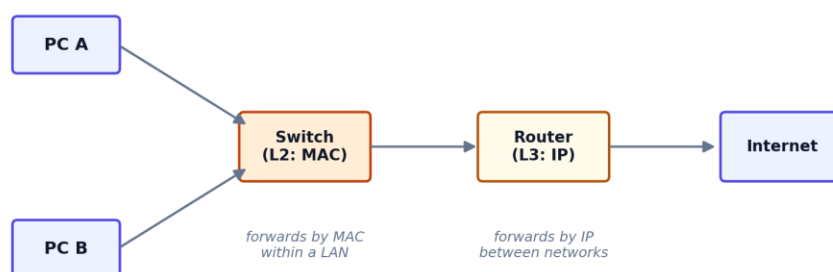
andar tak pahunchta hai.

8. Devices - hub vs switch vs router

● Must know

Three devices show up constantly, and the trick is to remember which layer each works at - that single fact answers most questions about them.

Which device works at which layer



Hub (L1) = dumb repeater, copies bits to every port. Switch (L2) = learns MACs, sends only to the right port. Router (L3) = connects different networks using IP.

A switch forwards by MAC inside a LAN. A router forwards by IP between different networks.

Device	Layer	Forwards by	Smart?
Hub / Repeater	1 Physical	Nothing - copies bits to every port	No, a dumb broadcaster
Switch / Bridge	2 Data Link	MAC address (within one LAN)	Learns which MAC is on which port
Router	3 Network	IP address (between networks)	Runs routing, connects networks

💡 Aha - Hub woh banda hai jo poori class mein chilla ke message bolta hai - sabko sunai deta hai. Switch smart hai, seedha us bande ke kaan mein bolta hai jise message dena hai. Router do alag buildings ke beech ka guard hai jo decide karta hai kaunsa message kaunsi building jaayega.

9. IP addressing - IPv4, classes, private, IPv6

● Must know

An IPv4 address is 32 bits, written as four numbers 0-255 separated by dots, like 192.168.1.10. That gives about 4.3 billion addresses (2^{32}), which the world has run out of - which is why IPv6 (128 bits) exists.

- IPv4 = 32 bits, 4 octets, e.g. 192.168.1.10. About 4.3 billion total.
- IPv6 = 128 bits, 8 groups of hex, e.g. 2001:db8::1. Effectively unlimited.
- Public IP = unique on the whole internet. Private IP = reused inside home / office networks, never routed on the public internet.
- Loopback = 127.0.0.1 (your own machine, localhost).

The class system (older, but interviewers still ask):

Class	First octet	Default mask	Use
A	0 - 127	/8 (255.0.0.0)	Very large networks
B	128 - 191	/16 (255.255.0.0)	Medium networks
C	192 - 223	/24 (255.255.255.0)	Small networks
D	224 - 239	-	Multicast
E	240 - 255	-	Experimental / reserved

The private ranges (RFC 1918) - memorize these three:

10.0.0.0/8 172.16.0.0/12 192.168.0.0/16 (plus 127.0.0.1 loopback and 169.254.0.0/16 link-local / APIPA).

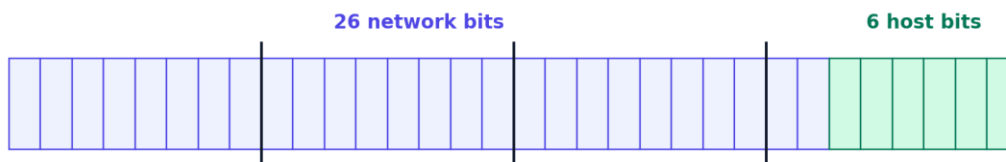
💡 Aha - Public IP tumhara ghar ka exact postal address hai jo poori duniya mein unique hai. Private IP ghar ke andar ke kamron ke number jaise hain - har ghar mein Room 1, Room 2 ho sakta hai, koi takraav nahi, kyunki bahar wale ko sirf ghar ka address dikhta hai, kamron ke number nahi.

10. Subnetting + CIDR - the math

🟢 Must know

A subnet splits one big network into smaller ones. CIDR notation like /24 says how many leading bits are the network part - the rest are host bits. The slash number is the count of network bits out of 32.

Subnetting: a /26 splits the 32 bits into network + host



192.168.1.0 / 26 mask 255.255.255.192

$$\text{Usable hosts} = 2^{(32-26)} - 2 = 2^6 - 2 = 62$$

Subtract 2: one for the network address (all host bits 0), one for the broadcast address (all host bits 1).

A /26 uses 26 network bits and leaves 6 host bits, so $2^6 - 2 = 62$ usable hosts.

The one formula to know cold:

Usable hosts = $2^{(32 - \text{prefix})} - 2$. Subtract 2 because the first address (all host bits 0) is the network address and the last (all host bits 1) is the broadcast address - neither can be assigned to a device.

CIDR	Subnet mask	Host bits	Usable hosts
/24	255.255.255.0	8	$2^8 - 2 = 254$
/25	255.255.255.128	7	$2^7 - 2 = 126$
/26	255.255.255.192	6	$2^6 - 2 = 62$
/27	255.255.255.224	5	$2^5 - 2 = 30$
/30	255.255.255.252	2	$2^2 - 2 = 2$ (point-to-point links)

💡 Aha - Subnetting ek bade hostel ko floors mein baant-ne jaisa hai. /24 matlab poora floor ek group, /26 matlab us floor ko 4 wings mein baant diya. Jitne zyada network bits, utne chhote groups par utne zyada groups ban sakte hain. Do address har group mein reserved - ek naam-plate (network), ek announcement (broadcast).

Common port numbers worth memorizing

Interviewers like to fire a quick 'which port does X use'. These are the ones that actually come up:

Service	Port	Transport
HTTP	80	TCP
HTTPS	443	TCP
DNS	53	UDP (TCP for large / zone transfer)

Service	Port	Transport
DHCP	67 server, 68 client	UDP
SSH	22	TCP
SMTP	25	TCP
FTP	20 data, 21 control	TCP
Telnet	23	TCP

Well-known ports are 0 to 1023 (the standard services above). Registered ports run 1024 to 49151, and ephemeral / dynamic ports (the temporary ones your client picks for an outgoing connection) run 49152 to 65535.

11. TCP vs UDP

● Must know

Both live at the Transport layer. TCP is the careful courier that sets up a connection, numbers every byte, waits for acknowledgements, and resends what is lost. UDP just fires packets off and hopes for the best - no setup, no guarantee, but almost no overhead.

Aspect	TCP	UDP
Connection	Connection-oriented (handshake first)	Connectionless (just send)
Reliability	Reliable - acks + retransmission	Unreliable - no acks, may drop
Ordering	In-order delivery guaranteed	No ordering guarantee
Speed	Slower (more overhead)	Faster (minimal overhead)
Header size	20 bytes minimum	8 bytes fixed
Flow / congestion control	Yes	No
Used by	HTTP(S), FTP, SMTP, SSH	DNS, DHCP, video / VoIP, games

💡 Aha - TCP registered post hai - signature leta hai, kho gaya toh dobara bhejta hai, par thoda slow. UDP normal postcard hai - phenk diya, pahuncha toh pahuncha. Live match ya video call mein ek frame chhoot jaaye toh chalega, par ruk-ruk ke aaye toh nahi - isliye woh UDP use karte hain.

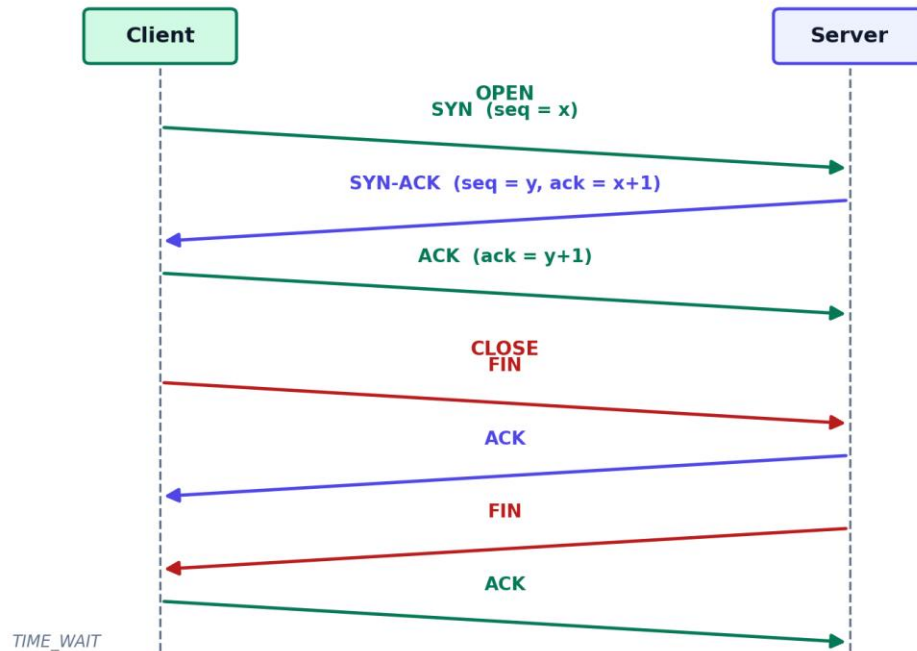
The interview follow-up: why does DNS use UDP? Because a lookup is one small request and one small reply - setting up a TCP connection for that would cost more than the query itself. DNS falls back to TCP only for large responses or zone transfers.

12. TCP connection - handshake + teardown

● Must know

Before TCP sends data, both sides agree to talk and exchange starting sequence numbers. That takes three messages - the 3-way handshake. Closing takes four, because each direction is shut down separately.

TCP: 3-way handshake to open, 4-way to close



Open: SYN, SYN-ACK, ACK. Close: FIN, ACK, FIN, ACK. The client ends in TIME_WAIT to catch any late packets.

The 3-way handshake (open):

7. SYN - client says "I want to talk, my sequence number starts at x".
8. SYN-ACK - server replies "ok, I heard x, my sequence starts at y".
9. ACK - client confirms "got your y". Now both sides are synced and data flows.

Why three and not two? Because both sides must confirm that the other can both send and receive. Two messages would only prove one direction works.

The 4-way teardown (close):

- FIN from the side that is done sending, ACK from the other.
- Then the second side sends its own FIN, and the first ACKs it.
- Four messages because closing one direction does not close the other - each side says "I am done" separately.

💡 Aha - 3-way handshake phone call shuru karne jaisa hai: "Hello, sunai de raha hai?" - "Haan sunai de raha hai, tumhe?" - "Haan mujhe bhi". Teen baar bolna padta hai taaki dono ko pakka ho jaaye ki line dono taraf clear hai.

13. Beyond the basics

Everything above is the fresher core - the part most rounds actually test. What follows is depth: congestion control, sliding-window protocols, the TLS handshake, and the supporting protocols. Strong candidates know these; they turn a passing answer into a confident one. If you are short on time before a round, skim this and focus on the §28 revision card.

14. Flow control vs congestion control

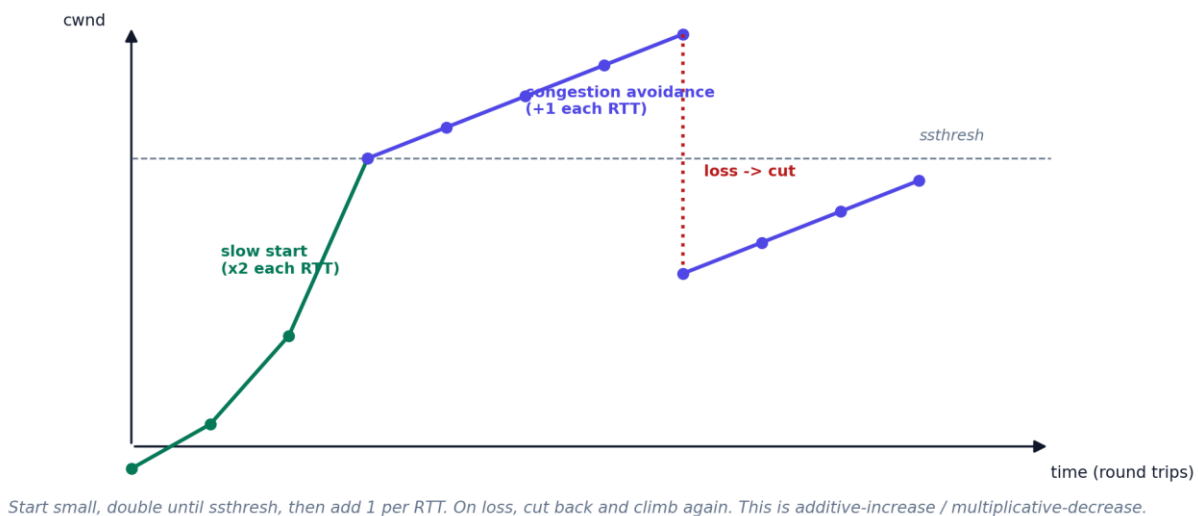
Good to know

Two different problems that students mix up. Flow control protects the receiver from being overwhelmed. Congestion control protects the network in the middle from being overwhelmed. TCP does both.

	Flow control	Congestion control
Protects	The receiver	The network (routers / links)
Signal	Receiver's advertised window (rwnd)	Packet loss / delay
Mechanism	Sliding window sized by the receiver	cwnd: slow start + AIMD

Congestion control is the famous one. TCP starts with a small congestion window and doubles it every round trip (slow start) until it hits a threshold, then grows by just one per round trip (congestion avoidance). On packet loss it cuts back sharply and climbs again. This pattern is called AIMD - additive increase, multiplicative decrease.

TCP congestion control: slow start (exponential) then AIMD (linear)



Start small, double until ssthresh, then add 1 per RTT. On loss, cut back and climb again. This is additive-increase / multiplicative-decrease.

Slow start doubles the window each RTT; congestion avoidance adds one; a loss cuts it back. Climb fast, back off hard.

💡 **Aha - Congestion control naye driver jaisa hai jo khaali road pe dheere se speed badhata hai - pehle double-double, phir ek-ek karke. Jaise hi traffic (packet loss) dikhta hai, ekdum brake maar ke speed aadhi kar deta hai, phir se dheere badhata hai. Flow control alag hai - woh saamne wale ki capacity dekhta hai, road ki nahi.**

15. Sliding window protocols

Rare / deep dive

How does a sender keep the pipe full without overwhelming the receiver? It sends a window of packets before waiting for acks, then slides the window forward as acks arrive. Three classic schemes:

Protocol	Window	On error	Efficiency
Stop-and-Wait	1 packet at a time	Resend the one packet	Low (idle while waiting)
Go-Back-N	N unacked allowed	Resend from the lost packet onward	Better, but re-sends good packets
Selective Repeat	N unacked allowed	Resend only the lost packet	Best, but needs receiver buffering

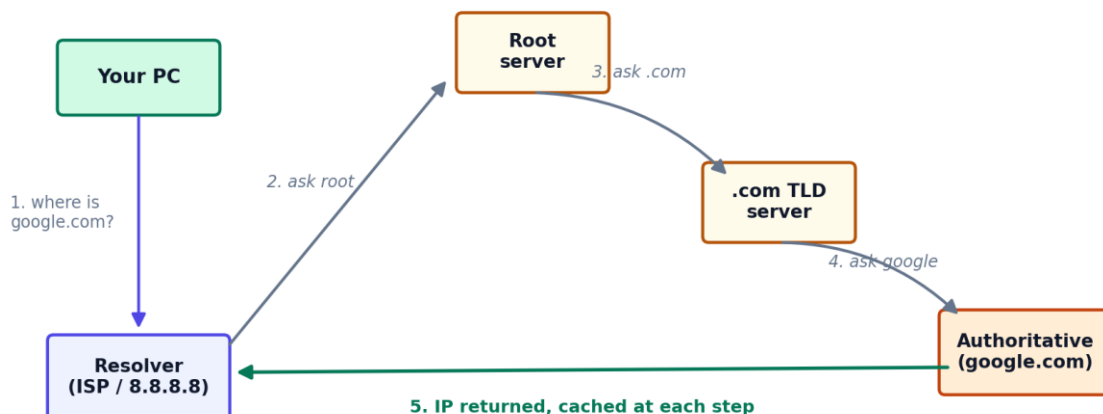
 **Aha - Stop-and-Wait matlab ek question bhejo, jawab ka wait karo, phir agla - bahut slow. Go-Back-N matlab ek galti pe us point se aage sab dobara bhejo. Selective Repeat matlab sirf galat wala dobara bhejo - smartest, par dono taraf zyada yaad rakhna padta hai.**

16. DNS - name to IP

Must know

Humans remember google.com; machines need 142.250.x.x. DNS is the phone book that translates. When a name is not cached, a recursive resolver walks the hierarchy: it asks a root server, which points to the .com TLD server, which points to google's authoritative server, which gives the final IP. Each step is cached so the next lookup is instant.

DNS resolution: turning google.com into an IP



The recursive resolver does the legwork. Caching at browser / OS / resolver means most lookups never reach the root.

The resolver does the legwork: root, then TLD, then authoritative. Caching means most lookups never reach the root.

Record types worth knowing:

Record	Maps	Example use
A	Name to IPv4	google.com to 142.250.0.0
AAAA	Name to IPv6	google.com to 2001:db8::1
CNAME	Name to another name (alias)	www to the root domain
MX	Domain to mail server	Where email for the domain goes
NS	Domain to its nameservers	Who is authoritative
PTR	IP to name (reverse)	Reverse lookup

DNS runs on port 53, over UDP for normal queries (small and fast) and over TCP for zone transfers or responses larger than 512 bytes.

💡 Aha - DNS phone-book hai. Tumhe naam yaad hai (google.com) par call lagane ke liye number chahiye (IP). Resolver woh dost hai jo tumhare liye directory mein dhoondh ke number nikal deta hai, aur agli baar ke liye yaad bhi rakh leta hai.

17. HTTP + HTTPS

🟢 Must know

HTTP is the request-response protocol the web speaks. The client sends a request with a method (the verb) and a path; the server replies with a status code and a body. HTTP is stateless - each request stands alone, which is why cookies and tokens exist to carry identity across requests.

The methods:

Method	Does	Safe?	Idempotent?
--------	------	-------	-------------

Method	Does	Safe?	Idempotent?
GET	Read a resource	Yes	Yes
POST	Create / submit	No	No
PUT	Replace a resource	No	Yes
PATCH	Partially update	No	Not guaranteed
DELETE	Remove a resource	No	Yes

Idempotent means doing it twice has the same effect as doing it once (DELETE the same row twice = still gone). Safe means no side effects (GET never changes data).

Status code classes:

Class	Means	Common ones
1xx	Informational	100 Continue
2xx	Success	200 OK, 201 Created, 204 No Content
3xx	Redirection	301 Moved Permanently, 302 Found, 304 Not Modified
4xx	Client error	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found
5xx	Server error	500 Internal Server Error, 502 Bad Gateway, 503 Unavailable

HTTP versions, briefly:

- HTTP/1.1 - one request at a time per connection, but connections stay open (persistent).
- HTTP/2 - multiplexing (many requests over one connection at once) + header compression. Much faster.
- HTTP/3 - runs over QUIC (which is built on UDP) to avoid TCP's head-of-line blocking.

HTTPS is just HTTP wrapped in TLS, on port 443 instead of 80. The TLS layer encrypts everything so no one in the middle can read or tamper with it.

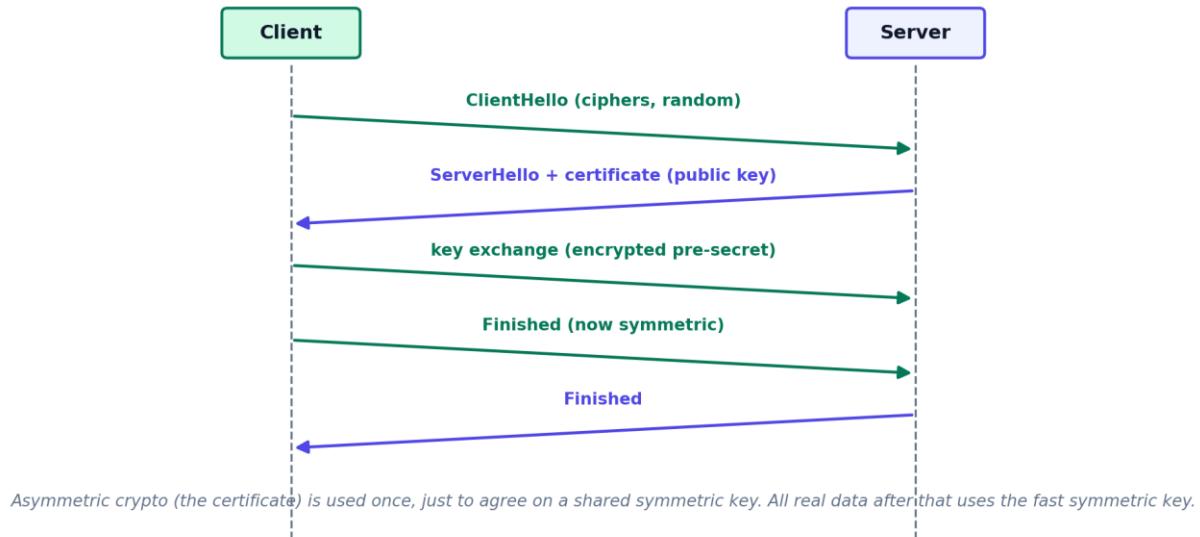
💡 **Aha - HTTP postcard pe likha message hai - koi bhi padh le. HTTPS wahi message ek locked box mein hai jiski chaabi sirf tumhare aur server ke paas hai. Stateless ka matlab server har request ko ajnabi ki tarah treat karta hai - isliye cookie ya token se batana padta hai ki "main wahi banda hoon".**

18. TLS handshake

🔗 Good to know

How do two strangers agree on a secret key over an open wire that anyone can watch? TLS uses asymmetric crypto once to bootstrap a fast symmetric key. The client and server exchange hellos, the server proves its identity with a certificate (containing its public key), they agree on a shared session key, and from then on all data uses that fast symmetric key.

TLS handshake: how HTTPS sets up a secure channel



Asymmetric crypto (the certificate) is used once, just to agree on a symmetric session key. The real data uses the fast symmetric key.

💡 Aha - TLS aisa hai jaise tum aur dukaandaar bina kisi ke samjhe ek secret code decide kar lo, sabke saamne baith ke. Certificate dukaandaar ka verified ID card hai jisse pata chalta hai woh asli hai, fraud nahi. Code set hone ke baad poori baat us code mein hoti hai - fast bhi, safe bhi.

19. ARP, DHCP, NAT, ICMP

🔗 Good to know

Four supporting protocols that round out almost every CN round:

- ARP (Address Resolution Protocol) - you know a device's IP but need its MAC to actually deliver on the local link. ARP shouts "who has this IP?" and the owner replies with its MAC.
- DHCP (Dynamic Host Configuration Protocol) - hands out IP addresses automatically when you join a network. The four steps spell DORA: Discover, Offer, Request, Acknowledge. Ports 67 (server) and 68 (client).
- NAT (Network Address Translation) - lets many private devices share one public IP. Your router rewrites the source address (and port) on the way out and reverses it on the way back. This is why home devices use 192.168.x.x but the world sees one public IP.
- ICMP (Internet Control Message Protocol) - the network's error and diagnostics messenger. ping and traceroute are built on it.

💡 Aha - ARP = "is IP wale ka ghar number 5 pe kaun rehta hai?" puchhna. DHCP = naye mehmaan ko aate hi ek room number de dena. NAT = poore ghar ka ek hi gate number, andar chaahe kitne kamre hon. ICMP = chowkidaar jo bolta hai "raasta band hai" ya "woh banda nahi mila".

20. What happens when you type google.com

● Must know

This single question touches almost every CN topic, which is exactly why interviewers love it. Narrate it in order:

What happens when you type google.com and hit Enter



DNS gives the IP, TCP opens the pipe, TLS secures it, HTTP asks for the page, the server replies, and the browser paints it. Every interview loves this question.

The seven-step journey. DNS finds the IP, TCP opens the pipe, TLS secures it, HTTP fetches the page, the browser renders.

10. Parse the URL - the browser pulls out the scheme (https), the host (google.com), and the path.
11. DNS lookup - resolve google.com to an IP, checking browser, OS, and resolver caches before asking the DNS hierarchy.
12. TCP handshake - open a connection to that IP on port 443 with SYN, SYN-ACK, ACK.
13. TLS handshake - agree on a secure session key so the data is encrypted.
14. HTTP request - send GET / with headers (host, cookies, user-agent).
15. Server responds - a status code (hopefully 200) plus the HTML, CSS, and JS.
16. Browser renders - parse the HTML, fetch linked resources (each may repeat steps 2-6), and paint the page.

💡 **Aha - Yeh ek pizza order karne jaisa hai. Address dhoondho (DNS), shop ko call lagao (TCP), secure line pe baat karo (TLS), order bolo (HTTP GET), shop pizza bhejti hai (response), aur tum table laga ke khaate ho (render). Ek sawaal mein poora networking ka syllabus aa jaata hai.**

21. Error detection + switching (brief)

● Rare / deep dive

Two short topics that occasionally come up:

- Error detection - the receiver needs to know if bits got flipped in transit. Parity bit (one bit, catches odd errors), checksum (sum of the data, used in TCP/IP), and CRC (a polynomial division, used in Ethernet frames) all detect errors. Hamming code can even correct single-bit errors.

- Switching - how data moves through the network. Circuit switching reserves a dedicated path end to end (old phone calls). Packet switching breaks data into packets that each find their own way (the internet) - more efficient and resilient.

These are the not-fresher-required depth flagged in §13. Know the names and the one-line difference for each.

@rathoreadiitya

22. Comparison tables (the ones interviewers love)

TCP vs UDP

Aspect	TCP	UDP
Connection	Connection-oriented	Connectionless
Reliable	Yes (acks + resend)	No
Ordered	Yes	No
Header	20 bytes min	8 bytes
Use	Web, file transfer, email	DNS, streaming, games

OSI vs TCP/IP

Aspect	OSI	TCP/IP
Layers	7	4
Role	Reference / teaching model	What the internet actually runs
Top layers	Session + Presentation + Application	One Application layer

Hub vs switch vs router

	Hub	Switch	Router
Layer	1 Physical	2 Data Link	3 Network
Forwards by	Broadcasts bits	MAC address	IP address
Scope	One collision domain	One LAN	Between networks

IPv4 vs IPv6

Aspect	IPv4	IPv6
Size	32 bits	128 bits
Notation	Decimal, dotted (192.168.1.1)	Hex, colons (2001:db8::1)
Address space	~4.3 billion	Effectively unlimited
Header	20 bytes min, variable	40 bytes fixed

Flow control vs congestion control

Aspect	Flow control	Congestion control
Protects	The receiver	The network
Driven by	Receiver window (rwnd)	Packet loss (cwnd)
Method	Sliding window	Slow start + AIMD

Symmetric vs asymmetric encryption

Aspect	Symmetric	Asymmetric
Keys	One shared key	Public + private key pair
Speed	Fast	Slow
Use in TLS	Bulk data after handshake	Agreeing on the shared key

23. Interview triggers - if they ask X, say Y

If they ask / say	You say
Difference between TCP and UDP	TCP is connection-oriented, reliable, ordered. UDP is connectionless and fast with no guarantees
Why is the handshake 3-way	Both sides must confirm they can send AND receive. Two messages prove only one direction
Why does DNS use UDP	A lookup is one small request and reply. A TCP setup would cost more than the query itself
What happens when you type a URL	DNS lookup, TCP handshake, TLS, HTTP request, response, render - walk it in order
Hosts in a /26	$2^{(32-26)} - 2 = 62$. Subtract the network and broadcast addresses
Switch vs router	A switch forwards by MAC inside one LAN. A router forwards by IP between networks
What makes HTTPS secure	TLS: an asymmetric handshake agrees on a symmetric key, then all data is encrypted
HTTP is stateless, so how do logins work	Cookies or tokens carry identity, since each request is independent
Difference between flow and congestion control	Flow control protects the receiver. Congestion control protects the network
What is ARP for	Mapping a known IP to the MAC needed to deliver on the local link

24. Common mistakes + traps

- Calling the TCP open a 4-way handshake. Opening is 3-way. Closing is 4-way.
- Saying UDP is reliable because "it still mostly arrives". UDP gives no delivery, ordering, or duplicate guarantees at all.
- Forgetting to subtract 2 in the host count. A /26 has 64 addresses but only 62 usable.
- Mixing up flow control and congestion control. Receiver vs network - keep them separate.

- Saying DNS is always UDP. It is UDP for normal queries, TCP for zone transfers and large responses.
- Confusing a switch and a router. Switch = MAC = within a LAN. Router = IP = between networks.
- Thinking HTTPS uses only asymmetric encryption. Asymmetric is used once to set up a symmetric key, which does the bulk work.
- Saying IPv6 exists for speed. It exists because IPv4's ~4.3 billion addresses ran out.

@rathoreadiitya

25. Model interview answers - say it like this

The 50-question bank below is for rapid recall. This section is the opposite: the 17 questions a CN round actually opens with, each with a short 15-second version for a quick round and a full answer you can speak in 20 to 40 seconds. Every answer follows the same shape - define it in one line, give one concrete example, then handle the follow-up before they ask. Read these out loud once; that is what makes them stick.

How to use a model answer: lead with the one-line definition so the interviewer knows you know it, then ground it in ONE example, then stop. Rambling past the example is how strong answers turn average. The follow-up line is the question they ask next - have it ready and you look two steps ahead.

Q. Walk me through the OSI model.

Short version (15 sec): 7 layers, each with one job: Application (HTTP, DNS), Presentation (encryption), Session (connections), Transport (TCP/UDP, ports), Network (IP, routing), Data Link (MAC, frames), Physical (bits). Mnemonic top-down: All People Seem To Need Data Processing.

Full version: OSI is a 7-layer reference model for how data moves across a network, each layer with one job. From the top: Application is where the user protocols live, like HTTP and DNS; Presentation handles encryption and formatting; Session manages connections between apps; Transport gives end-to-end delivery with TCP or UDP and ports; Network handles logical addressing and routing with IP; Data Link frames data and uses MAC addresses; Physical is the raw bits on the wire. A handy mnemonic top-down is All People Seem To Need Data Processing.

Follow-up they hit you with: Which layer does a router work at, and a switch? A router is Layer 3 (Network, IP); a traditional switch is Layer 2 (Data Link, MAC).

Q. OSI vs TCP/IP model?

Short version (15 sec): OSI is the 7-layer theory model; TCP/IP is the 4-layer model the internet actually runs on. TCP/IP merges OSI's top three layers into one Application layer and the bottom two into Network Access.

Full version: Both are layered models, but OSI is the 7-layer theoretical reference, while TCP/IP is the 4-layer model the internet actually runs on. TCP/IP collapses OSI's top three layers, Application, Presentation, and Session, into a single Application layer, and merges Physical and Data Link into a Network Access layer. So TCP/IP is Application, Transport, Internet, and Network Access. OSI is the teaching tool; TCP/IP is the implementation.

Follow-up they hit you with: Why did TCP/IP win? It came with working protocols and the actual internet, while OSI stayed mostly a reference model.

Q. TCP vs UDP - when do you use which?

Short version (15 sec): TCP is connection-oriented and reliable - handshake, ordering, retransmission - use it for web, email, and files. UDP is connectionless and best-effort but fast - use it for video calls, streaming, games, and DNS.

Full version: Both are Transport-layer protocols but with opposite trade-offs. TCP is connection-oriented and reliable: it sets up a connection with a handshake, guarantees ordered, error-checked delivery, and does flow and congestion control, so it is slower but dependable. UDP is connectionless and best-effort: no handshake, no ordering or retransmission guarantees, just fast lightweight datagrams. Use TCP when every byte must arrive, like web pages, email, or file transfer. Use UDP when speed beats completeness, like video calls, live streaming, online games, and DNS.

Weak answer (skip it): Saying UDP is just a worse TCP. UDP is the right choice when you want low latency and can tolerate a lost packet, like a video frame.

Follow-up they hit you with: Why is the TCP header bigger? TCP needs at least 20 bytes for sequence numbers, acknowledgements, and window size; UDP needs only 8 bytes because it tracks none of that.

Q. Explain the TCP 3-way handshake.

Short version (15 sec): Client sends SYN, server replies SYN-ACK, client sends a final ACK. Three messages so both sides agree on starting sequence numbers and the connection is open.

Full version: It is how TCP opens a reliable connection and syncs sequence numbers. The client sends a SYN with its initial sequence number. The server replies with a SYN-ACK: it acknowledges the client's SYN and sends its own SYN with its sequence number. The client sends a final ACK for the server's SYN. Now both sides have agreed on starting sequence numbers and the connection is established. Three messages, hence three-way.

Follow-up they hit you with: Why not two-way? Both sides must sync sequence numbers and confirm the other can both send and receive; two messages would only confirm one direction.

Q. Why does TCP teardown take 4 steps when setup takes 3?

Short version (15 sec): Closing is per-direction because TCP is full-duplex. Each side sends its own FIN and gets its own ACK, so that is four messages, while setup can combine the server's SYN and ACK into one.

Full version: Because closing is independent for each direction. A TCP connection is full-duplex, so each side closes its own half separately. One side sends FIN, the other ACKs it, but that side may still have data to send, so its own FIN comes later as a separate message, and the first side ACKs that. That is four messages. In the handshake the server can combine its SYN and ACK into one packet, but in teardown the ACK and the FIN usually cannot be combined because the second side is not done sending yet.

Follow-up they hit you with: What is the TIME_WAIT state? After the final ACK the closer waits for twice the maximum segment lifetime, so any delayed packets die out before the port is reused.

Q. What happens when you type google.com and press Enter?

Short version (15 sec): DNS resolves the name to an IP, TCP connects on port 443 with the 3-way handshake, TLS sets up encryption, the browser sends an HTTP GET, the server returns HTML, and the browser renders it and fetches the CSS, JS, and images.

Full version: First the browser resolves the name: it checks its cache, then asks a DNS resolver, which walks from a root server to the .com TLD server to Google's authoritative server to get the IP. Then it opens a TCP connection to that IP on port 443 with the 3-way handshake, and does a TLS handshake for HTTPS. Then it sends an HTTP GET request; the server returns the HTML; the browser parses it and fetches CSS, JS, and images, then renders the page. ARP and routing move the packets hop by hop underneath all of this.

Follow-up they hit you with: Where does ARP fit in? When a packet must go to the next hop on the local network, ARP resolves that next hop's IP to its MAC address so the frame can be addressed.

Q. How does DNS resolution work?

Short version (15 sec): Your machine checks its cache, then asks a recursive resolver, which walks root to the .com TLD server to the authoritative server to get the IP, caches it for the TTL, and hands it back.

Full version: DNS turns a human name like google.com into an IP address. Your machine first checks its local cache, then asks a recursive resolver, usually run by your ISP. If the resolver does not have it cached, it queries the hierarchy: a root server points it to the .com TLD server, the TLD server points it to Google's authoritative name server, and that returns the actual IP. The resolver caches the answer for its TTL and hands it back to you.

Follow-up they hit you with: Does DNS use TCP or UDP? Mostly UDP on port 53 for speed, but it falls back to TCP for large responses like zone transfers or when a reply exceeds the UDP size limit.

Q. HTTP vs HTTPS, and what does TLS add?

Short version (15 sec): HTTP sends plaintext; HTTPS is HTTP over TLS, which adds encryption, authentication via the server certificate, and integrity. HTTPS is port 443, plain HTTP is port 80.

Full version: HTTP is the protocol browsers use to request and receive web pages, but it sends everything in plaintext, so anyone in between can read or tamper with it. HTTPS is HTTP running over TLS, which adds encryption so the data is unreadable in transit, authentication so you know you are talking to the real server via its certificate, and integrity so tampering is detected. HTTPS uses port 443; plain HTTP uses port 80.

Follow-up they hit you with: Is TLS symmetric or asymmetric encryption? Both: the handshake uses asymmetric keys to securely agree on a shared secret, then the actual data uses fast symmetric encryption with that secret.

Q. Flow control vs congestion control?

Short version (15 sec): Flow control stops a fast sender from overwhelming a slow receiver, using the receiver's advertised window. Congestion control stops senders from overwhelming the network, using the congestion window.

Full version: Both stop a sender from sending too fast, but they protect different things. Flow control stops a fast sender from overwhelming a slow RECEIVER, and TCP does it with the receiver's advertised window, which says how much buffer space is free. Congestion control stops senders from overwhelming the NETWORK in the middle, and TCP does it with a congestion window managed by slow start and additive-increase, multiplicative-decrease. So flow control is end-to-end about the receiver; congestion control is about the shared network.

Weak answer (skip it): Treating them as the same thing. One protects the receiver's buffer, the other protects the network from collapse.

Follow-up they hit you with: What does the sender actually send? The minimum of the receiver window and the congestion window, so whichever limit is tighter wins.

Q. Hub vs switch vs router?

Short version (15 sec): Hub is Layer 1 and just floods every port. Switch is Layer 2 and forwards a frame by MAC to the right port. Router is Layer 3 and forwards packets between different networks by IP.

Full version: They operate at different layers. A hub is a Layer 1 device that just repeats incoming bits out of every port, so it floods everything and shares bandwidth - basically obsolete. A switch is a Layer 2 device that learns MAC addresses and forwards a frame only to the correct port, so it is far more efficient within a local network. A router is a Layer 3 device that connects different networks and forwards packets between them using IP addresses and a routing table.

Follow-up they hit you with: Which one gives you separate collision domains? A switch - each port is its own collision domain, while a hub puts everyone in one.

Q. Explain IP addressing - public vs private, IPv4 vs IPv6.

Short version (15 sec): IPv4 is 32 bits, about 4.3 billion addresses that ran out. Private ranges (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16) are reused locally; public addresses are globally unique. IPv6 is 128 bits with practically unlimited space.

Full version: An IP address identifies a device on a network. IPv4 is 32 bits, written as four numbers like 192.168.1.1, giving about 4.3 billion addresses, which ran out. Private ranges (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16) are reused inside local networks and never routed on the public internet; public addresses are globally unique. IPv6 is 128 bits, written in hexadecimal, and gives a practically unlimited address space, which is the long-term fix for exhaustion.

Follow-up they hit you with: How do private addresses reach the internet then? Through NAT, which translates many private addresses to a shared public one at the router.

Q. What is subnetting and CIDR?

Short version (15 sec): Subnetting splits a network by borrowing host bits for the network part. CIDR writes the split as a slash, like /24 = 256 addresses, 254 usable after the network and broadcast addresses.

Full version: Subnetting splits one large network into smaller logical networks by borrowing bits from the host part of the address for the network part. CIDR notation writes the split as a slash number, like /24, meaning the first 24 bits are the network and the rest is for hosts. A /24 gives 256 total addresses, of which 254 are usable because one is the network address and one is the broadcast. Subnetting improves organisation, security, and routing efficiency.

Follow-up they hit you with: How many usable hosts in a /26? 64 total minus 2, so 62 usable hosts.

Q. What is ARP and why is it needed?

Short version (15 sec): ARP maps a Layer 3 IP to a Layer 2 MAC on the local network. A device broadcasts 'who has this IP', the owner replies with its MAC, and the sender caches the mapping.

Full version: ARP, the Address Resolution Protocol, maps a Layer 3 IP address to a Layer 2 MAC address within a local network. You need it because once a packet reaches the local network, the actual frame delivery uses MAC addresses, not IP. So a device broadcasts an ARP request asking who has this IP, the owner replies with its MAC, and the sender caches that mapping in its ARP table to address the frame.

Follow-up they hit you with: What is the difference from DNS? DNS maps a name to an IP; ARP maps an IP to a MAC. Different layers, different jobs.

Q. What does DHCP do?

Short version (15 sec): DHCP auto-assigns IP config when a device joins, via DORA: Discover, Offer, Request, Acknowledge. The lease includes the IP, subnet mask, default gateway, and DNS server, and it renews over time.

Full version: DHCP automatically assigns IP configuration to a device when it joins a network, so nobody sets addresses by hand. The exchange is four steps, DORA: the client broadcasts a Discover, servers reply with an Offer of an address, the client sends a Request for one offer, and the server sends an Acknowledge to lease it. The lease includes the IP, subnet mask, default gateway, and DNS server, and it expires and renews over time.

Follow-up they hit you with: What if the DHCP server is down? The device may fall back to a self-assigned link-local address (APIPA, 169.254.x.x) and cannot reach the wider network.

Q. What is NAT and why do we use it?

Short version (15 sec): NAT lets many private devices share one public IP. The router rewrites the private source address on outgoing packets and tracks the mapping by port, so replies return to the right device. It conserves scarce IPv4 addresses.

Full version: NAT, Network Address Translation, lets many devices on a private network share one public IP address. The router rewrites the private source address on outgoing packets to its public address and tracks the mapping, so replies come back to the right internal device, usually by port number. The main reason is conserving the scarce IPv4 public addresses, and it also hides the internal network as a side benefit.

Follow-up they hit you with: How does the router know which internal device a reply belongs to? It keeps a translation table keyed by the public port it assigned to each connection - that is PAT, port-based NAT.

Q. Walk me through HTTP methods and status codes.

Short version (15 sec): Methods state intent: GET reads, POST creates, PUT replaces, PATCH partially updates, DELETE removes. Status codes: 2xx success, 3xx redirect, 4xx client error, 5xx server error.

Full version: HTTP methods state the intent of a request: GET reads data, POST creates, PUT replaces, PATCH partially updates, and DELETE removes. Status codes report the result in ranges: 2xx is success, like 200 OK; 3xx is redirection, like 301 Moved Permanently; 4xx is a client error, like 404 Not Found or 401 Unauthorized; and 5xx is a server error, like 500 Internal Server Error. The first digit tells you the category instantly.

Follow-up they hit you with: Difference between PUT and POST? PUT is idempotent - sending it twice leaves the same result - while POST can create a new resource each time, so it is not idempotent.

Q. Stop-and-Wait vs Go-Back-N vs Selective Repeat?

Short version (15 sec): Stop-and-Wait sends one frame and waits for its ACK. Go-Back-N sends a window and resends the lost frame plus every one after it. Selective Repeat sends a window but resends only the lost frame, buffering the rest.

Full version: These are sliding-window protocols for reliable delivery, differing in how much is in flight and how errors are handled. Stop-and-Wait sends one frame and waits for its ACK before the next, which is simple but slow. Go-Back-N sends a window of frames, and if one is lost it retransmits that frame and every one after it, so the receiver discards out-of-order frames. Selective Repeat also sends a window but retransmits only the specific lost frame, buffering the out-of-order ones, which is more efficient but needs more receiver memory.

Follow-up they hit you with: Which does TCP resemble most? TCP is closest to Selective Repeat in spirit, using selective acknowledgements so only the missing segments are resent.

@rathoreaditiya

26. Frequently seen in placement interviews (2025-26)

These keep coming up in placement and off-campus rounds, grouped by where they land. Same syllabus, asked the way real interviewers ask it. If a row feels shaky, go back to that section before your round.

Where it shows up	What they actually ask
Models + layers	Walk through the OSI 7 layers and what each does. OSI vs TCP/IP. Which layer a router, switch and hub work at.
TCP vs UDP	TCP vs UDP and when you pick each. Explain the 3-way handshake and why not 2-way. The 4-way teardown and why TIME_WAIT exists.
Addressing + subnetting	For 192.168.1.0/26, how many hosts and what is the broadcast address. Public vs private IP. What CIDR fixed over classes.
DNS + web	What happens when you type google.com and press enter. How DNS resolves a name. HTTP vs HTTPS and what TLS adds.
Supporting protocols	What ARP does and when. The DHCP DORA steps. What NAT solves. Flow control vs congestion control.
Whiteboard + practical	Subnetting numericals live on paper. Read a status code (301 vs 302, 401 vs 403). Name the protocol from its port number.

💡 Aha - Ek baat pakki hai: subnetting numerical aur 'type google.com' wali full journey, in dono mein se ek to aata hi hai. Inhe ratt lo, bolke practice karo.

Where these come from: [GFG Computer Networks interview questions](#) and the [CN last-minute notes](#), cross-checked against recent off-campus experiences.

27. Interview FAQ bank - 50 questions

Quick-recall once the model answers above are solid. One or two lines each; if you can answer all 50 fast, your CN fundamentals are ready.

Question	Answer
What is a computer network?	A set of devices connected to share data and resources using agreed protocols.
What is a protocol?	A set of rules that governs how devices format, transmit, and receive data.
Name the OSI layers top to bottom.	Application, Presentation, Session, Transport, Network, Data Link, Physical.
What is the PDU at each layer?	Data (upper layers), Segment (Transport), Packet (Network), Frame (Data Link), Bits (Physical).
How many layers in the TCP/IP model?	Four: Application, Transport, Internet, and Link / Network access.
OSI vs TCP/IP?	OSI is a 7-layer teaching reference. TCP/IP is the 4-layer model the internet actually runs on.
What is encapsulation?	Each layer wrapping the data from the layer above in its own header (and the Data Link layer adds a trailer).
Hub vs switch vs router?	Hub broadcasts bits (L1). Switch forwards by MAC within a LAN (L2). Router forwards by IP between networks (L3).
Difference between MAC and IP address?	A MAC is a fixed hardware address for local delivery. An IP is a logical address used for routing across networks.
What is ARP?	Address Resolution Protocol: it maps a known IP to the MAC address needed to deliver on the local link.
How big is an IPv4 address?	32 bits, written as four octets, giving about 4.3 billion addresses.
How big is an IPv6 address?	128 bits, written as eight hex groups, giving an effectively unlimited space.
Why does IPv6 exist?	IPv4's roughly 4.3 billion addresses ran out. IPv6's 128 bits solve the exhaustion.
Name the private IP ranges.	10.0.0.0/8, 172.16.0.0/12, and 192.168.0.0/16 (RFC 1918).
What is 127.0.0.1?	The loopback address (localhost) - your own machine.
What does /24 mean in CIDR?	The first 24 bits are the network part, leaving 8 host bits (mask 255.255.255.0).
How many usable hosts in a /26?	$2^{(32-26)} - 2 = 62$. Subtract the network and broadcast addresses.
Why subtract 2 in the host count?	The all-zero host address is the network address and the all-one is the broadcast address - neither is assignable.
TCP vs UDP?	TCP is connection-oriented, reliable, and ordered. UDP is connectionless, fast, and gives no guarantees.
Which apps use UDP?	DNS, DHCP, video / VoIP, and online games - where speed beats perfect delivery.
TCP header size?	20 bytes minimum, up to 60 with options.
UDP header size?	8 bytes, fixed.
Describe the TCP 3-way handshake.	SYN from the client, SYN-ACK from the server, ACK from the client. Then data flows.
Why is the handshake 3-way and not 2?	Both sides must confirm they can send and receive. Two messages would prove only one direction.
Why does closing a TCP connection take 4 steps?	Each direction is closed independently: FIN and ACK each way.

What is TIME_WAIT?	A short wait after closing so any late or duplicate packets are handled before the port is reused.
What is flow control?	Stopping a fast sender from overwhelming a slow receiver, using the receiver's advertised window.
What is congestion control?	Stopping senders from overwhelming the network, using cwnd with slow start and AIMD.
What is slow start?	TCP starting with a small congestion window and doubling it each round trip until a threshold.
What is AIMD?	Additive Increase, Multiplicative Decrease: grow the window by one per RTT, halve it on loss.
Stop-and-Wait vs Go-Back-N vs Selective Repeat?	One packet at a time; resend from the lost packet onward; resend only the lost packet.
What is DNS?	The system that translates a domain name like google.com into an IP address.
Which port does DNS use?	Port 53: UDP for normal queries, TCP for zone transfers and large responses.
Walk through DNS resolution.	Check caches, then the recursive resolver asks a root server, then the TLD server, then the authoritative server.
What is an A record?	A DNS record mapping a domain name to an IPv4 address (AAAA is for IPv6).
What is a CNAME record?	An alias mapping one domain name to another name.
What is an MX record?	It points a domain to its mail server.
What is HTTP?	A stateless request-response protocol the web uses, on port 80.
What does stateless mean?	Each request is independent - the server keeps no memory of previous requests by itself.
HTTP methods?	GET, POST, PUT, PATCH, DELETE, HEAD, OPTIONS.
Which HTTP methods are idempotent?	GET, PUT, DELETE, and HEAD. POST is not; PATCH is not guaranteed.
Meaning of 200, 301, 404, 500?	200 OK, 301 Moved Permanently, 404 Not Found, 500 Internal Server Error.
HTTP/1.1 vs HTTP/2?	HTTP/1.1 is one request at a time per connection. HTTP/2 multiplexes many over one connection and compresses headers.
What is HTTP/3 built on?	QUIC, which runs over UDP, to avoid TCP's head-of-line blocking.
What is HTTPS?	HTTP over TLS on port 443, so the traffic is encrypted and authenticated.
Describe the TLS handshake.	Hellos, the server's certificate (public key), agreement on a symmetric session key, then encrypted data.
Symmetric vs asymmetric encryption?	Symmetric uses one shared key (fast). Asymmetric uses a public / private pair (slow) and bootstraps the symmetric key.
What is DHCP?	It auto-assigns IP addresses via DORA: Discover, Offer, Request, Acknowledge. Ports 67 and 68.
What is NAT?	Network Address Translation: many private devices share one public IP by rewriting addresses and ports at the router.
What happens when you type a URL and press Enter?	DNS lookup, TCP handshake, TLS handshake, HTTP request, server response, then the browser renders the page.

28. The 30-minute revision card

Memorize cold: OSI is 7 layers (All People Seem To Need Data Processing). TCP = reliable, ordered, 3-way open / 4-way close; UDP = fast, best-effort. Switch is Layer 2 (MAC), router is Layer 3 (IP). Usable hosts = $2^{(32-\text{prefix})} - 2$. Flow control protects the receiver, congestion control protects the network.

The numbers to know cold:

TCP header 20 bytes, UDP 8 bytes. IPv4 32 bits, IPv6 128 bits. Ports: HTTP 80, HTTPS 443, DNS 53, DHCP 67/68, SSH 22, SMTP 25. Usable hosts = $2^{(32-\text{prefix})} - 2$.

One-liners to fire back:

- OSI 7 layers top to bottom: Application, Presentation, Session, Transport, Network, Data Link, Physical.
- PDUs: data, segment, packet, frame, bits.
- TCP = connection-oriented, reliable, ordered, 20-byte header. UDP = connectionless, fast, 8-byte header.
- Handshake = SYN, SYN-ACK, ACK (open). Teardown = FIN, ACK, FIN, ACK (close).
- Why 3-way: both sides confirm send + receive.
- Switch = MAC = within a LAN (L2). Router = IP = between networks (L3).
- Private ranges: 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16.
- Flow control protects the receiver. Congestion control protects the network (slow start + AIMD).
- DNS: caches, then root, then TLD, then authoritative. Port 53, mostly UDP.
- HTTPS = HTTP + TLS on 443. Asymmetric sets up a symmetric key, symmetric does the bulk.
- URL journey: DNS, TCP, TLS, HTTP request, response, render.

If interviewer says X, you say Y:

- "Hosts in a /26" -> $2^6 - 2 = 62$.
- "Why DNS over UDP" -> small query, a TCP setup would cost more than the lookup.
- "What happens when you type a URL" -> DNS, TCP, TLS, HTTP, response, render.

Two traps that cost you easy points:

- Calling the TCP open a 4-way handshake - opening is 3-way, closing is 4-way.
- Forgetting to subtract 2 from the host count - a /26 has 64 addresses but 62 usable.

29. Final word

You have the OSI layers, the TCP vs UDP split, the handshake, a worked subnet calculation, DNS, and the full URL journey. A CN round often includes one "TCP vs UDP", one "what happens when you type a URL", and one addressing or subnetting question. You have already practised all three. Narrate them calmly and you will be ready for the vast majority of

@rathoreadiitya